ARCA: Adapter-Residual Credit Assignment When Token Signals Collapse

Anonymous Author(s)
Affiliation
Address
email

Abstract

Token-level credit assignment for language-model reinforcement learning is an active area of research, with methods that reweight policy-gradient credit using intrinsic signals such as surprisal, entropy, and divergence from a reference policy. At the same time, practical LLM-RL pipelines overwhelmingly rely on parameter-efficient fine-tuning, especially LoRA. These threads are usually studied separately: token-credit methods are specified as if the policy were fully trainable, while PEFT is treated as an implementation detail. We argue that this separation is misleading. Under LoRA, the policy is restricted to a low-rank neighbourhood of the reference, which can flatten the per-token differences used by common intrinsic credit signals and drive their normalized weights toward uniform broadcast. We formalize this failure mode and propose measuring it directly with concentration diagnostics such as weight Gini and effective-token ratio. We then introduce *Adapter-Residual Credit Assignment* (ARCA), a lightweight alternative that derives token salience from the adapter's own hidden-state residual, $\|h_t^{\text{adapted}} - h_t^{\text{base}}\|_2$. This signal asks where the adapter actually changes the model, rather than where the output distribution appears uncertain or shifted, and requires no reward model, value head, or tree construction. Our experiments are designed to test whether adapter-residual credit remains non-degenerate under LoRA and whether preserving that structure improves RL fine-tuning under matched rollout budgets.

1 Introduction

Reinforcement learning has become a central component of large language model (LLM) post-training, particularly for alignment and for reasoning-oriented tasks with verifiable rewards. A persistent challenge across these settings is *credit assignment*: trajectories are long, rewards are sparse and outcome-level, and it is not obvious how a single scalar signal at the end of a generation should be distributed across hundreds or thousands of token decisions. Recent work has responded with a rapidly growing toolbox of token-level credit-assignment methods, including entropy-aware modulation, process reward models, tree-based prefix values, and reward redistribution [7, 16, 32, 34, 9, 41, 14].

In parallel, essentially all open-source and academic LLM-RL training uses parameter-efficient fine-tuning, most commonly LoRA [11]. This is an empirical reality driven by memory and compute constraints: every widely used RL framework, from TRL to verl, uses LoRA by default, and recent results show that LoRA + RL can be dramatically more sample- and parameter-efficient than full fine-tuning [33]. Yet the two research threads are developed in near-total isolation. Papers on token-level credit assignment specify methods without reference to the adaptation strategy, and PEFT is treated as an orthogonal implementation detail.

In this work we argue that this separation is a mistake, and that the *interaction* between PEFT and credit assignment is itself a first-class scientific object. Intrinsic token-level weighting schemes derive

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

per-token salience from quantities such as surprisal, entropy reduction, or divergence between the policy and a reference model. Under LoRA, however, the policy is constrained to a small low-rank neighbourhood of the reference, and the per-token differences that these signals measure become approximately uniform across positions. We formalize this as a collapse of the salience distribution toward uniform broadcast, characterize it with Gini coefficient and effective token count, and show that the collapse reproduces across multiple output-distribution-based weighting rules. This gives a mechanistic explanation for the empirically observed failure of full-token GRPO + LoRA [15]: the weighting signal itself is degraded before training ever begins.

Rather than trying to recover per-token structure from signals that LoRA intrinsically flattens, we measure salience directly from the adapter's own contribution to the forward pass. Adapter-Residual Credit Assignment (ARCA) sets the salience at position t equal to the norm of the adapter residual, $\|h_t^{\text{adapted}} - h_t^{\text{base}}\|_2$, computed via a forward pass with the adapter disabled. This signal is positive whenever the adapter is active, non-uniform across positions because adapter input activations are non-uniform, and requires no additional networks, reward models, or tree construction. It reuses the same adapter-disabled forward pass already needed for KL regularization and divergence weighting.

Concretely, this paper makes three contributions:

1. We identify and formalize a PEFT-specific failure mode of intrinsic token weighting: under LoRA, surprisal-, entropy-, and divergence-derived salience can collapse toward uniform token credit even when the underlying RL objective is unchanged.
2. We introduce ARCA, a lightweight adapter-residual credit assignment rule whose normalized token weights remain non-degenerate whenever the adapter has position-varying hidden-state impact.
3. We provide a diagnostic evaluation framework that reports both downstream reward metrics and concentration metrics, making it possible to separate methods that merely define token weights from methods whose weights retain usable signal under the adaptation regime actually used in LLM-RL.

The remainder of the paper develops these contributions. Section 2 presents the weighting schemes, explains why output-distribution signals collapse under LoRA, and defines ARCA. Section 3 reports the diagnostic and performance comparison on MATH with Qwen3-1.7B under the seven-run submission-time sweep. Extended related work and theoretical interpretation appear in Appendices A and D.

2 Methods

We consider on-policy reinforcement learning for autoregressive language models with trajectory-level rewards. Let π_θ denote a language model parameterized by θ, generating a completion $y = (y_1, \dots, y_T)$ conditioned on a prompt x. After sampling a full trajectory, the model receives a scalar reward $R(x, y) \in \mathbb{R}$ computed by an external verifier, such as exact-answer correctness or unit-test pass rate. Our objective is

$$J(\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}[R(x, y)], \tag{1}$$

where $\mathcal{D}$ denotes the prompt distribution. We focus on the regime most relevant to recent reasoning-model training: sparse outcome rewards, on-policy sampling, and no learned value function.

2.1 Policy Gradient with Trajectory-Level Reward

For a fixed prompt x, the standard REINFORCE estimator is

$$g_{\text{rf}}(x, y) = R(x, y) \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(y_t \mid y_{<t}, x). \tag{2}$$

In practice, a baseline $b(x)$ is subtracted to reduce variance without changing the expected gradient as long as $b(x)$ does not depend on the sampled action at token t:

$$g_{\text{base}}(x, y) = (R(x, y) - b(x)) \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(y_t \mid y_{<t}, x). \tag{3}$$

We use prompt-level multi-sample baselines. Given K sampled completions $\{y^{(i)}\}_{i=1}^{K}$ for the same prompt x with rewards $\{R^{(i)}\}_{i=1}^{K}$:

$$b_{\mathrm{RLOO}}^{(i)}(x) = \frac{1}{K-1} \sum_{j \neq i} R^{(j)} \tag{4}$$

is the leave-one-out baseline used in RLOO-style estimators, while GRPO-style normalization can be written as

$$A_{\mathrm{GRPO}}^{(i)}(x) = \frac{R^{(i)} - \mu_R(x)}{\sigma_R(x) + \varepsilon}, \tag{5}$$

where $\mu_R(x)$ and $\sigma_R(x)$ are the within-prompt mean and standard deviation of the K rewards. In both cases, the resulting scalar advantage is then broadcast uniformly over all tokens in the sampled trajectory. This scalar-broadcast assumption is precisely what we relax.

2.2 Token-Level Credit Redistribution

We introduce token-level weights $w_t(x, y)$ that redistribute a trajectory-level advantage across tokens:

$$g_w(x, y) = A(x, y) \sum_{t=1}^{T} w_t(x, y) \, \nabla_\theta \log \pi_\theta(y_t \mid y_{<t}, x), \tag{6}$$

where $A(x, y)$ denotes either $R(x, y) - b_{\mathrm{RLOO}}(x)$ or $A_{\mathrm{GRPO}}(x, y)$ depending on the baseline choice. The weights satisfy

$$\sum_{t=1}^{T} w_t(x, y) = 1, \qquad w_t(x, y) \geq 0. \tag{7}$$

This normalization keeps the total update magnitude comparable across weighting schemes and isolates the effect of credit redistribution from simple rescaling.

Uniform token credit corresponds to $w_t = 1/T$ for all t. Our goal is to replace this uniform allocation with *intrinsic* weights derived from quantities already produced by the policy during generation. We do not train token-level reward models, estimate prefix values, or build explicit search trees.

2.3 Intrinsic Token-Weighting Mechanisms

We define each method through an unnormalized salience score $\alpha_t(x, y) \geq 0$ and then normalize within each trajectory:

$$w_t(x, y) = \frac{\alpha_t(x, y) + \varepsilon}{\sum_{k=1}^{T} (\alpha_k(x, y) + \varepsilon)}, \tag{8}$$

with a small $\varepsilon > 0$ to avoid degenerate all-zero cases. In implementation, the weights are treated as *detached* scalars when multiplying the policy-gradient term, so the update does not introduce second-order derivatives through the weighting function itself. This keeps the optimization rule close in spirit and cost to standard critic-free policy gradients.

Uniform Weighting (Baseline).

$$w_t^{\text{uniform}} = \frac{1}{T}. \tag{9}$$

Combined with an RLOO or GRPO baseline, this recovers the standard critic-free estimator that assigns identical credit to every token in the sampled completion.

Surprisal Weighting. Our first intrinsic score is token surprisal,

$$\alpha_t^{\text{surp}}(x, y) = -\log \pi_\theta(y_t \mid y_{<t}, x). \tag{10}$$

This emphasizes low-probability decisions under the current policy. Intuitively, these are tokens where the model commits to a less routine continuation and where uniform credit assignment may be most diluted by predictable filler tokens.

Entropy-Reduction Weighting. Our second intrinsic score measures local entropy reduction. Let

$$H_t^{\mathrm{pre}}(x, y) = -\sum_v \pi_\theta(v \mid y_{<t}, x) \log \pi_\theta(v \mid y_{<t}, x) \tag{11}$$

denote the predictive entropy before sampling token y_t, and let

$$\Delta H_t^{\mathrm{ent}}(x, y) = \max\left(0, H_t^{\mathrm{pre}}(x, y) - H_{t+1}^{\mathrm{pre}}(x, y)\right) \tag{12}$$

for $t < T$. For the final token we set $\Delta H_T^{\mathrm{ent}} = 0$. We then use

$$\alpha_t^{\mathrm{ent}}(x, y) = \Delta H_t^{\mathrm{ent}}(x, y). \tag{13}$$

This score highlights tokens after which the model's next-step distribution becomes substantially more concentrated. Such events often correspond to commitment points in reasoning, where one branch of continuation becomes much more likely than its alternatives.

Policy-Divergence Weighting. A third intrinsic score measures how much the current policy has drifted from a reference π_{ref} (typically the base model prior to RL) at each token position:

$$\alpha_t^{\mathrm{div}}(x, y) = |\log \pi_\theta(y_t \mid y_{<t}, x) - \log \pi_{\mathrm{ref}}(y_t \mid y_{<t}, x)|. \tag{14}$$

In the RLHF/RLVR setting π_{ref} is already available because it is used to compute the KL regularizer, so this score is essentially free.

Length-Robust Normalization. Because reasoning trajectories can vary substantially in length, all weights are normalized within each sampled completion rather than across the minibatch. This ensures that longer trajectories do not automatically receive larger aggregate updates simply because they contain more token positions with nonzero salience. The comparison between weighting schemes therefore reflects *where* credit is assigned within a trajectory, not how much total credit a longer sequence receives.

Batch-Level Baselines. All weighting schemes can be paired with either RLOO or GRPO-style prompt-level baselines. Unless otherwise specified, we use the same rollout groups, optimizer, sampling hyperparameters, and baseline family across methods. This isolates token redistribution as the only algorithmic difference.

2.4 Signal Collapse Under Low-Rank Adaptation

So far our exposition has treated the policy π_θ as an unconstrained language model being updated by gradient descent. In practice, however, almost all LLM-RL pipelines apply LoRA [11], which restricts π_θ to a rank-r perturbation of a frozen reference policy π_{ref}. We now show that the intrinsic weighting schemes defined above are qualitatively degraded by this constraint.

Notation. Write the logits produced by the base model at position t as $z_t^{\mathrm{ref}} = W_{\mathrm{lm}} h_t^{\mathrm{base}}$ and the logits produced by the adapted model as $z_t^\theta = W_{\mathrm{lm}} h_t^{\mathrm{adapted}}$, where $h_t^{\mathrm{adapted}} = h_t^{\mathrm{base}} + \Delta h_t$ and Δh_t is the sum of LoRA adapter contributions propagated through the transformer to position t. Each adapter contributes $B_\ell A_\ell x_{\ell,t}$ at layer ℓ, where $B_\ell \in \mathbb{R}^{d \times r}$ and $A_\ell \in \mathbb{R}^{r \times d}$, so the raw per-layer perturbation at each position lies in the fixed r-dimensional column space of B_ℓ.

Collapse of surprisal and entropy. Because $\|\Delta h_t\|$ is bounded by a product of adapter norms and input activation norms that are themselves roughly stationary across positions in a well-trained base model, the first-order perturbation to per-token surprisal,

$$-\log \pi_\theta(y_t \mid y_{<t}, x) = -\log \pi_{\mathrm{ref}}(y_t \mid y_{<t}, x) + O(\|\Delta h_t\|),$$

is dominated by the base model's surprisal pattern, which is task-agnostic and primarily reflects filler versus content tokens. The same holds for predictive entropy. Consequently, when we normalize these scores within a trajectory (equation (8)), the resulting weights are nearly identical to the base model's surprisal or entropy weights, regardless of what the adapter has learned.

Collapse of policy divergence. Divergence weighting, $\alpha_t^{\text{div}} = |\log \pi_\theta(y_t) - \log \pi_{\text{ref}}(y_t)|$, is of direct interest because it quantifies exactly the policy change that the RL update is trying to drive. Under LoRA, however, this quantity is small and approximately uniform across positions for a different reason: the KL budget $\text{KL}(\pi_\theta \| \pi_{\text{ref}})$ is bounded by the rank-r adapter's capacity, so the per-token log-probability ratios are compressed toward zero [2]. Under the within-trajectory normalization in equation (8), this compression maps any approximately constant score function to the uniform distribution, giving $w_t^{\text{div}} \approx 1/T$.

Consequence. We summarize the practical implication as follows. Let the Gini coefficient $G(w)$ and the effective-token ratio $\text{EffN}(w)/T = 1/(T \sum_t w_t^2)$ be standard concentration measures, with $G(w) = 0$ and $\text{EffN}/T = 1$ corresponding to perfectly uniform weights. Then for surprisal, entropy-reduction, and divergence weighting under LoRA, both concentration measures approach their uniform limits as the adapter rank r decreases relative to the hidden dimension d. In this regime, "intrinsic token weighting" is indistinguishable from uniform broadcast, and intrinsic-weighting methods cannot provide the credit-redistribution benefit they were designed to deliver. This is *not* a problem with the underlying signals; it is a structural consequence of applying them inside a low-rank adaptation.

2.5 Adapter-Residual Credit Assignment (ARCA)

The collapse result motivates a different design choice. Rather than measuring per-token properties of the *output distribution* (whose shape is dominated by the frozen base model under LoRA), we measure per-token properties of the *adapter's own contribution* to the forward pass.

Definition. Given a model with a LoRA adapter, let h_t^{adapted} denote the last-layer hidden state at position t with the adapter enabled, and h_t^{base} the same hidden state with the adapter disabled. Define the Adapter-Residual Credit Assignment salience as

$$\alpha_t^{\text{ARCA}}(x, y) = \|h_t^{\text{adapted}} - h_t^{\text{base}}\|_2, \tag{15}$$

and normalize within the trajectory as in equation (8) to obtain per-token weights w_t^{ARCA}. The adapter residual is computed once per minibatch via an extra no-grad forward pass with the adapter disabled, using `model.disable_adapter()` in frameworks such as PEFT. This is exactly the same call already required to compute a KL regularizer against the reference policy or to support divergence weighting, so ARCA adds no new infrastructure.

Why ARCA does not collapse. The key property that distinguishes ARCA from the intrinsic schemes above is that it measures a quantity whose *per-token* value is actively shaped by the attention and MLP pathways through which the adapter input activations are routed. Even for a low-rank adapter with small $\|B_\ell A_\ell\|$, the input activations $x_{\ell,t}$ are non-uniform across positions (attention patterns, layer norms, and content-versus-filler distinctions are already non-uniform in the base model), so $\|\Delta h_t\|$ retains non-trivial position-to-position variation. Formally, as the adapter rank goes to zero the signal α_t^{ARCA} vanishes uniformly, but its *normalized* counterpart w_t^{ARCA} remains non-degenerate as long as the adapter is nontrivially active at *any* position.

Interpretation as implicit gradient-informed credit. ARCA can be read as a cheap proxy for a gradient-based notion of token importance. Positions at which the adapter contribution is large are positions at which the adapter's parameter gradient has high contribution to the loss, and therefore positions where a policy-gradient update can have the most effect. This is a substantially weaker statement than the one made by a learned critic, but it has the advantage of being available for free from quantities the adapted forward pass already computes, and of being exactly targeted at the PEFT-specific failure mode introduced above.

3 Experiments

We evaluate whether adapter-residual credit assignment gives a useful token-level signal in the low-rank fine-tuning regime. Because the submission-time experiments are intentionally compact, the empirical section focuses on a single controlled sweep: one model, one task, one advantage estimator,

Table 1: **Submission-time sweep.** All runs use `Qwen/Qwen3-1.7B-Base`, MATH, GRPO, one seed, and 100 training steps. The non-ARCA baselines are matched at LoRA rank 64; ARCA is evaluated at three ranks to separate credit assignment from adapter capacity.

Method	LoRA rank	Trainable params	Role in comparison
Uniform	64	*tbd*	Same-rank baseline
Surprisal	64	*tbd*	Intrinsic token baseline
Entropy reduction	64	*tbd*	Intrinsic token baseline
Policy divergence	64	*tbd*	Intrinsic token baseline
ARCA	4	*tbd*	Low-rank ARCA control
ARCA	16	*tbd*	Mid-rank ARCA control
ARCA	64	*tbd*	Same-rank ARCA comparison

and seven weighting configurations. This design trades breadth for a cleaner test of the central claim: ARCA changes credit assignment without adding trainable parameters beyond the LoRA adapter used by the matched baselines.

3.1 Setup

Model and task. We fine-tune `Qwen/Qwen3-1.7B-Base` on the MATH training split and evaluate on held-out MATH examples. Prompts ask the model to solve the problem step by step and place the final answer in `\boxed{}`. Rewards are binary exact-match scores computed from the final boxed answer after normalization.

Training. All runs use GRPO prompt-level advantages, LoRA adapters on the attention and MLP projections, and the same rollout budget, optimizer, learning-rate schedule, prompt format, and decoding parameters. Each training step samples $K = 4$ completions per prompt, uses 100 update steps, and evaluates greedy accuracy and pass@4 on 200 held-out MATH examples. We run a single seed, 1337. Checkpoints are saved every 20 steps. We use AdamW with learning rate 5×10^{-6}, zero weight decay, a 10-step warmup followed by cosine decay, batch size 2, gradient accumulation over 4 microbatches, maximum prompt length 512, maximum generation length 512, temperature 1.0, and top-p 0.95 sampling during training.

Methods. The sweep contains seven runs. Four baselines use LoRA rank $r = 64$: uniform token weighting, surprisal weighting, entropy-reduction weighting, and policy-divergence weighting. The proposed method, ARCA, is run at $r \in \{4, 16, 64\}$. The $r = 64$ comparison tests ARCA against baselines with the same trainable parameter count. The $r = 4$ and $r = 16$ comparisons test whether ARCA remains competitive with fewer adapter parameters than the $r = 64$ baselines.

3.2 Main Results on MATH

Table 2 reports held-out MATH performance after RL fine-tuning. The primary comparison is ARCA at rank 64 versus the rank-64 baselines, which holds the trainable parameter count fixed. The rank-4 and rank-16 ARCA runs provide a more stringent control: if lower-rank ARCA is competitive with rank-64 baselines, the gain cannot be attributed simply to using a larger adapter.

3.3 Credit-Assignment Diagnostics

Performance alone does not show whether a method changes token-level credit assignment. We therefore log the concentration of token weights during training. We report the Gini coefficient $G(w)$ and effective-token ratio $\mathrm{EffN}(w)/T$. Uniform weighting has $G = 0$ and $\mathrm{EffN}/T = 1$; highly concentrated weights have G near one and a small effective-token ratio.

The diagnostic prediction is not that the most concentrated scheme should always win. Rather, the useful regime is a middle ground: weights should depart from uniform enough to express token-level credit, but should not collapse onto a tiny number of tokens. This is the regime ARCA is designed to target. The rank ablation also tests a possible alternative explanation: ARCA introduces no additional

Table 2: **Held-out MATH performance.** Greedy accuracy and pass@4 after GRPO fine-tuning. Numbers are placeholders until the seven runs complete.

Method	LoRA rank	Greedy acc.	Pass@4	Train reward
Start checkpoint	–	*tbd*	*tbd*	–
Uniform	64	*tbd*	*tbd*	*tbd*
Surprisal	64	*tbd*	*tbd*	*tbd*
Entropy reduction	64	*tbd*	*tbd*	*tbd*
Policy divergence	64	*tbd*	*tbd*	*tbd*
ARCA	4	*tbd*	*tbd*	*tbd*
ARCA	16	*tbd*	*tbd*	*tbd*
ARCA	64	*tbd*	*tbd*	*tbd*

Table 3: **Token-weight diagnostics.** Concentration statistics averaged over training. These diagnostics test whether a method produces a non-uniform but non-collapsed token-credit signal. Numbers are placeholders until the runs complete.

Method	LoRA rank	$G(w)$	EffN$/T$
Uniform	64	*tbd*	*tbd*
Surprisal	64	*tbd*	*tbd*
Entropy reduction	64	*tbd*	*tbd*
Policy divergence	64	*tbd*	*tbd*
ARCA	4	*tbd*	*tbd*
ARCA	16	*tbd*	*tbd*
ARCA	64	*tbd*	*tbd*

trainable parameters, so a same-rank gain would reflect the loss weighting, while a low-rank ARCA gain would be inconsistent with a simple parameter-count explanation.

3.4 Implementation and Metrics

The main experiment entrypoint is `experiment/hf/run_experiment.py`. For each minibatch, the script samples completions from the current policy, computes binary rewards after full generation, constructs GRPO advantages, and applies the weighted token-level objective from Section 2. Surprisal uses current-policy token log probabilities, entropy-reduction uses next-token entropy drops, policy-divergence compares adapted and adapter-disabled log probabilities, and ARCA weights tokens by adapter-residual norms in the final hidden layer.

The code records per-step reward mean, reward variance, loss, completion length, token-weight Gini, effective-token ratio, and, for ARCA, the mean and maximum adapter-residual norm. Final evaluation records greedy accuracy and pass@4 on held-out MATH examples. These metrics allow us to distinguish three claims: whether the method trained, whether it changed token-credit structure, and whether that structure improved held-out performance.

3.5 Compute

Each run fits on a single H100-class GPU. The seven submission-time runs are executed independently, one configuration per GPU, with wall-clock training time on the order of a few hours per run plus final evaluation. The reported sweep therefore requires seven single-GPU runs; preliminary development runs were used to tune runtime and checkpointing but are not included in the reported comparison.

3.6 Broader Impacts

This work is methodological and studies how to assign token-level credit during reinforcement learning for language models. The potential positive impact is improved sample efficiency and interpretability of RL fine-tuning, especially in settings where practitioners already use parameter-efficient adaptation. The same techniques could also improve the fine-tuning of models for harmful or misleading generation if applied without appropriate task, data, and deployment safeguards. We do not release a new general-purpose model in this paper.

3.7 Limitations

This sweep is deliberately small: one model, one dataset, one seed, one advantage estimator, and seven configurations. It is sufficient to test the parameter-matched ARCA comparison and the low-rank control, but it does not establish robustness across seeds, tasks, or algorithms. We view the results as a targeted validation of the mechanism rather than a broad benchmark claim.

References

[1] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2024.

[2] Anonymous. LoRA as an implicit KL regularizer in GRPO fine-tuning: From theory to practice, 2026. Under review at Transactions on Machine Learning Research.

[3] Meng Cao, Shuyuan Zhang, Xiao-Wen Chang, and Doina Precup. SCAR: Shapley credit assignment for more efficient RLHF, 2025.

[4] Yekun Chai, Haoran Sun, Huang Fang, Shuohuan Wang, Yu Sun, and Hua Wu. MA-RLHF: Reinforcement learning from human feedback with macro actions. In *The Thirteenth International Conference on Learning Representations*, 2025.

[5] Hongzhan Chen, Tao Yang, Shiping Gao, Ruijun Chen, Xiaojun Quan, Hongtao Tian, and Ting Yao. Discriminative policy optimization for token-level reward models. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 9546–9565. PMLR, 2025.

[6] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems 30*, 2017.

[7] Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Yuchen Zhang, Jiacheng Chen, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, Jiarui Yuan, Huayu Chen, Kaiyan Zhang, Xingtai Lv, Shuo Wang, Yuan Yao, Xu Han, Hao Peng, Yu Cheng, Zhiyuan Liu, Maosong Sun, Bowen Zhou, and Ning Ding. Process reinforcement through implicit rewards, 2025.

[8] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025.

[9] Yuhang He, Haodong Wu, Siyi Liu, Hongyu Ge, Hange Zhou, Keyi Wu, Zhuo Zheng, Qihong Lin, Zixin Zhong, and Yongqi Zhang. Rethinking token-level credit assignment in RLVR: A polarity-entropy analysis, 2026.

[10] Falko Helm, Nico Daheim, and Iryna Gurevych. Token weighting for long-range language modeling. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1440–1459, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics.

[11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[12] Jian Hu, Jason Klein Liu, Haotian Xu, and Wei Shen. REINFORCE++: Stabilizing critic-free policy optimization with global advantage normalization, 2025.

[13] Miaobo Hu, BoKun Wang, Shuhao Hu, Ruohan Wang, Xin Wang, Xiaobo Guo, Daren Zha, and Jun Xiao. PSPO: Trainable potential-based reward shaping with internal model signals for post-training policy optimization of large language models, 2026. ICLR 2026 submission, rejected.

[14] Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux. VinePPO: Refining credit assignment in RL training of LLMs, 2024.

[15] Alan Lee and Harry Tong. Token-efficient RL for LLM reasoning, 2025.

[16] Jiahui Li, Lin Li, Tai-Wei Chang, Kun Kuang, Long Chen, Jun Zhou, and Cheng Yang. Red: Unleashing token-level rewards from holistic feedback via reward redistribution, 2024.

[17] Shiwei Li, Xiandi Luo, Haozhao Wang, Xing Tang, Ziqiang Cui, Dugang Liu, Yuhua Li, Xiuqiang He, and Ruixuan Li. Beyond higher rank: Token-wise input-output projections for efficient low-rank adaptation, 2025.

[18] Yingru Li, Jiawei Xu, Ziniu Li, Jiacai Liu, Wei Liu, Yuxuan Tong, Longtao Zheng, Zhenghai Xue, Yaxiang Zhang, Tianle Cai, Ge Zhang, Qian Liu, and Baoxiang Wang. The optimal token baseline: Variance reduction for long-horizon LLM-RL, 2026.

[19] Ziniu Li, Tian Xu, Yushun Zhang, Zhihang Lin, Yang Yu, Ruoyu Sun, and Zhi-Quan Luo. ReMax: A simple, effective, and efficient reinforcement learning method for aligning large language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*. PMLR, 2024.

[20] Haoming Meng, Kexin Huang, Shaohang Wei, Chiyu Ma, Shuo Yang, Xue Wang, Guoyin Wang, Bolin Ding, and Jingren Zhou. Sparse but critical: A token-level analysis of distributional shifts in RLVR fine-tuning of LLMs, 2026.

[21] Julian Minder, Clément Dumas, Stewart Slocum, Helena Casademunt, Cameron Holmes, Robert West, and Neel Nanda. Narrow finetuning leaves clearly readable traces in activation differences, 2025.

[22] Youssef Mroueh, Nicolas Dupuis, Brian Belgodere, Apoorva Nitsure, Mattia Rigotti, Kristjan Greenewald, Jiri Navratil, Jerret Ross, and Jesus Rios. Revisiting group relative policy optimization: Insights into on-policy and off-policy training, 2025.

[23] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35*, 2022.

[24] Prasanna Parthasarathi, Mathieu Reymond, Boxing Chen, Yufei Cui, and Sarath Chandar. Grpo-λ: Credit assignment improves llm reasoning, 2025.

[25] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1889–1897, Lille, France, 07–09 Jul 2015. PMLR.

[26] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.

[27] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.

[28] Zikang Shan, Han Zhong, Liwei Wang, and Li Zhao. Bringing value models back: Generative critics for value modeling in LLM reinforcement learning, 2026.

[29] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024.

[30] Lloyd S. Shapley. A value for n-person games. In Harold W. Kuhn and Albert W. Tucker, editors, *Contributions to the Theory of Games II*, pages 307–317. Princeton University Press, Princeton, 1953.

[31] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning to summarize from human feedback. In *Advances in Neural Information Processing Systems 33*, 2020.

[32] Hieu Tran, Zonghai Yao, and Hong Yu. Exploiting tree structure for credit assignment in RL training of LLMs, 2025.

[33] Shangshang Wang, Julian Asilis, Ömer Faruk Akgül, Enes Burak Bilgin, Ollie Liu, and Willie Neiswanger. Tina: Tiny reasoning models via LoRA, 2025.

[34] Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xiong-Hui Chen, Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji Song, Bowen Yu, Gao Huang, and Junyang Lin. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for LLM reasoning, 2025.

[35] Xumeng Wen, Zihan Liu, Shun Zheng, Shengyu Ye, Zhirong Wu, Yang Wang, Zhijian Xu, Xiao Liang, Junjie Li, Ziming Miao, Jiang Bian, and Mao Yang. Reinforcement learning with verifiable rewards implicitly incentivizes correct reasoning in base LLMs, 2025.

[36] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, may 1992.

[37] Wei Xiong, Jiarui Yao, Yuhui Xu, Bo Pang, Lei Wang, Doyen Sahoo, Junnan Li, Nan Jiang, Tong Zhang, Caiming Xiong, and Hanze Dong. A minimalist approach to LLM reasoning: from rejection sampling to reinforce, 2025.

[38] Shentao Yang, Shujian Zhang, Congying Xia, Yihao Feng, Caiming Xiong, and Mingyuan Zhou. Preference-grounded token-level guidance for language model fine-tuning. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10–16, 2023*, 2023.

[39] Qingyu Yin, Yulun Wu, Zhennan Shen, Sunbowen Li, Zhilin Wang, Yanshu Li, Chak Tou Leong, Jiale Kang, and Jinjin Gu. Evaluating parameter efficient methods for RLVR, 2025.

[40] Hee Suk Yoon, Eunseop Yoon, Mark A. Hasegawa-Johnson, Sungwoong Kim, and Chang D. Yoo. ConfPO: Exploiting policy model confidence for critical token selection in preference optimization. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 72641–72655. PMLR, 2025.

[41] Song Yu, Li Li, Wenwen Zhao, and Zhisheng Yang. ERPO: Token-level entropy-regulated policy optimization for large reasoning models, 2026.

[42] Chong Zhang, Yue Deng, Xiang Lin, Bin Wang, Dianwen Ng, Hai Ye, Xingxuan Li, Yao Xiao, Zhanfeng Mo, Qi Zhang, and Lidong Bing. 100 days after deepseek-r1: A survey on replication studies and more directions for reasoning language models, 2025.

[43] Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy optimization, 2025.

[44] Han Zhong, Zikang Shan, Guhao Feng, Wei Xiong, Xinle Cheng, Li Zhao, Di He, Jiang Bian, and Liwei Wang. DPO meets PPO: Reinforced token optimization for RLHF. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 78498–78521. PMLR, 2025.

A Related Work

A.1 PEFT and Low-Rank Adaptation in Language RL

Parameter-efficient fine-tuning, and in particular LoRA [11], has become the default adaptation strategy for essentially all open-source LLM-RL work. Tina demonstrated that tiny LoRA adapters on a 1.5B base model suffice to reach DeepSeek-R1-class reasoning behaviour when combined with GRPO, at a tiny fraction of full fine-tuning cost [33]. A broader systematic evaluation of PEFT methods under RLVR [39] benchmarks over a dozen PEFT variants (including DoRA, AdaLoRA, MiSS, PiSSA, MiLoRA, VeRA and Rank-1) on DeepSeek-R1-Distill models and finds that standard LoRA is not optimal—structural variants such as DoRA, AdaLoRA and MiSS consistently outperform it, and SVD-initialised variants (PiSSA, MiLoRA) suffer from *spectral collapse*, a finding that is broadly compatible with our own signal-collapse analysis. Token-Efficient RL introduces critic-free LoRA-compatible variants of GRPO (S-GRPO and T-SPMO) that focus training on a subset of tokens, reporting large gains on small models where full-token GRPO + LoRA trains unstably [15]; we now provide a mechanistic explanation for that instability via signal collapse. *LoRA as an Implicit KL Regularizer* analyses how LoRA restricts the policy to a rank-constrained neighbourhood of the reference, deriving an explicit rank-dependent upper bound on the KL divergence between policy and reference throughout training [2]. Our Section 2.4 builds directly on this observation: an implicit KL bound is the same mechanism that forces per-token log-probability differentials to be small and approximately uniform, which is the formal content of our collapse result.

On the analysis side, *Narrow Finetuning Traces* shows that narrow finetuning leaves clearly readable traces in the activation differences between base and finetuned hidden states, and that the finetuning domain can be recovered from those differences alone using simple diffing tools such as patchscopes and activation steering [21]. This is directly relevant to ARCA because the adapter residual $h_t^{\text{adapted}} - h_t^{\text{base}}$ is a per-position activation difference of exactly the kind they study; their results supply empirical evidence that this difference carries meaningful, semantically non-trivial content—the property our method exploits. Concurrent work on TopLoRA studies how to concentrate LoRA capacity on a small number of high-impact tokens via token-wise input-output projections [17]. This can be viewed as a complementary response to the same underlying problem: TopLoRA modifies the adapter itself so that its capacity is allocated more selectively, while ARCA leaves the adapter unchanged but reweights the policy gradient so that token updates reflect where the adapter actually had non-trivial influence. To our knowledge, no prior work has characterized the interaction between LoRA and token-level credit assignment, nor proposed an adaptation-aware intrinsic weighting scheme.

A.2 Positioning of This Work

The literature makes three points clear. First, critics are not strictly required for effective LLM post-training: critic-free estimators such as RLOO, ReMax, GRPO, and GSPO are already competitive in both RLHF and RLVR [1, 19, 29, 43]. Second, many researchers have concluded that trajectory-level rewards are too coarse and have introduced denser supervision through process reward models, redistribution rules, tree structures, optimal baselines, temporal traces, or entropy-based modulation [16, 7, 32, 3, 24, 18, 13, 9, 41, 20]. Third, the overwhelming majority of practical LLM-RL pipelines are *trained with LoRA*, and a focused sub-literature has studied PEFT-specific effects in this regime [11, 33, 39, 2, 15, 21, 17].

Our contribution is to bridge the second and third of these threads. Existing intrinsic credit-assignment signals (surprisal, entropy, divergence) are not novel as standalone objects, and we do not claim otherwise; they are our baselines. Our novel claim is that these signals *interact pathologically with the adaptation strategy that the field has converged on*: under LoRA they collapse toward uniform, so any paper that reports a null result for "fancy intrinsic weighting versus uniform" under LoRA is observing a LoRA artefact, not evidence against token-level credit assignment. We formalize this, supply a unified diagnostic (Gini/EffN) that makes the collapse visible at training time, and propose ARCA as an adaptation-aware alternative whose construction directly avoids the failure mode we identify.

B Extended Related Work

B.1 From RLHF to RLVR

Modern language-model post-training inherits its optimization machinery from policy-gradient reinforcement learning, from REINFORCE [36] to trust-region and clipped-surrogate methods such as TRPO and PPO [25, 27]. Advantage estimation, especially GAE, became the standard variance-reduction device in continuous-control RL by learning value functions over states or prefixes [26]. In language modeling, these ideas entered mainstream use through RLHF systems that optimize sequence-level rewards derived from human preferences or reward models [6, 31, 23].

The recent reasoning-model wave shifted part of the field from preference-based RLHF toward reinforcement learning with verifiable rewards (RLVR), where supervision is often a deterministic correctness signal on math or code tasks. DeepSeekMath introduced GRPO as a practical critic-free alternative for these settings [29], and DeepSeek-R1 popularized large-scale RL-only or RL-dominant reasoning pipelines [8]. Subsequent surveys document how quickly this regime became the default template for open reasoning-model replication and extension [42]. This transition matters for our setting because RLVR makes sparse outcome rewards and long reasoning trajectories central, thereby exposing the credit-assignment problem more directly than earlier short-form RLHF tasks.

B.2 Critic-Based and Critic-Free Policy Optimization

The classical solution to delayed reward is to learn a critic or value function and convert returns into token- or prefix-level advantages. PPO-style RLHF pipelines follow this recipe, but the value-estimation problem is particularly awkward for text generation because prefixes are high-dimensional, non-Markov, and semantically heterogeneous. VinePPO demonstrated that standard value heads produce poor estimates of expected returns for reasoning tasks and proposed Monte Carlo vine-style rollout estimates as a more accurate alternative [14]. GenAC extends this idea by replacing scalar value prediction with a generative critic that uses chain-of-thought reasoning for value estimation [28].

A growing body of work questions whether value heads are necessary at all in LLM post-training. ReMax replaces learned critics with a greedy baseline and shows that much of PPO's practical benefit can be retained with a simpler REINFORCE-style objective [19]. Back to Basics systematically compares PPO with RLOO-style estimators and finds that critic-free methods can match or exceed value-based baselines in RLHF [1]. REINFORCE++ further strengthens this line by replacing the prompt-level advantage normalization used by GRPO and RLOO with a global, batch-level normalization, stabilizing critic-free policy optimization without reintroducing a learned value function [12]. In reasoning-focused RLVR, GRPO normalizes rewards across a group of sampled responses rather than learning prefix values [29], and later work refines this family with theoretical analyses, off-policy variants, and sequence-level formulations such as GSPO [22, 43]. These methods substantially simplify the optimization stack, but they generally still broadcast a trajectory-level signal across all tokens once a scalar baseline is fixed.

B.3 Reasoning-Specific Analyses of RLVR

Another recent line asks why critic-free RLVR works so well for reasoning in the first place. A Minimalist Approach to LLM Reasoning shows that a simple rejection-sampling baseline (RAFT) is surprisingly competitive with GRPO and PPO on reasoning tasks, and that GRPO's advantage over vanilla REINFORCE comes primarily from discarding prompts whose sampled responses are all incorrect rather than from reward normalization; the authors distill this insight into Reinforce-Rej, a minimal REINFORCE variant that filters both entirely-correct and entirely-incorrect samples, suggesting that much of the field's complexity is optional rather than essential [37]. Complementarily, Wen et al. analyze RLVR theoretically and empirically, arguing that answer-level verifiable rewards can nonetheless incentivize correct intermediate reasoning early in a trajectory [35]. These results are important because they show that uniform outcome-level objectives already contain nontrivial reasoning signal.

At the same time, they do not eliminate the question of *which* tokens should absorb that signal. Recent empirical analysis of RLVR training dynamics suggests that improvements are disproportionately driven by a minority of high-entropy "forking" tokens, and that restricting updates to this subset can remain competitive or even outperform full-token updates in some settings [34]. Building on this observation, EAPO adapts conditional mutual information to the autoregressive RLVR setting and proves that the credit a token can carry is upper-bounded by its entropy, motivating an entropy-aware modulation of per-token learning signals [9]. ERPO similarly identifies "critical decision pivots" at high-entropy states and applies targeted exploration at those positions [41]. *Sparse but Critical* argues, via a distributional-shift analysis based on cross-sampling between policy and reference, that a small fraction of tokens accounts for most of the useful update magnitude in RLVR, and proposes a divergence-based selection rule for identifying them [20]. These entropy- and divergence-based methods are the closest concurrent work to the intrinsic weighting schemes that form our baselines; the present paper does *not* claim priority over them. Instead, our contribution is to show that their underlying signals are structurally degraded under LoRA, and to provide a credit-assignment mechanism (ARCA) that does not suffer the same degradation.

B.4 Fine-Grained Credit Assignment Beyond Uniform Token Updates

A large parallel literature attempts to make supervision denser than a single end-of-trajectory reward. One family learns explicit token- or process-level reward estimators. Preference-grounded Token-level Guidance derives token-level signals from preference data for fine-tuning [38], while Discriminative Policy Optimization learns token-level Q-style reward models from preferences without requiring fine-grained annotation [5]. PRIME pushes this direction further for reasoning tasks by constructing implicit process rewards and updating process reward models online from rollouts and outcome labels [7]. Reinforced Token Optimization (RTO) frames RLHF as a token-level MDP and uses DPO log-likelihood ratios as a per-token reward signal derived from the preference model, which is then optimized with PPO to produce fine-grained token-level credit [44]. These methods can produce rich token-level feedback, but they rely on an auxiliary reward-modeling component that our approach intentionally avoids.

A second family learns a small token-level critic on top of the policy's own hidden states (rather than an external reward model) and uses its predictions to form token-level advantages. This corresponds closely to an earlier version of our own framework, and we found through an extensive literature review that the token-level actor–critic design space was already well-covered: VinePPO [14] and GenAC [28] cover non-parametric and generative critics respectively, and OTB [18] provides a principled variance-minimizing position-dependent baseline. We therefore do *not* position our critic-based configuration as a proposed method; we include it purely as a strong within-family baseline in our comparisons, and acknowledge this prior art explicitly.

A third family redistributes outcome rewards algorithmically rather than by training a dense reward model. RED derives token-level rewards by redistributing holistic reward-model scores over a trajectory [16]. SCAR uses Shapley-inspired attribution to estimate token or span contributions from sequence-level feedback [3, 30]. TEMPO uses groups of sampled solutions to build a prefix tree and compute nonparametric prefix values, yielding branch-sensitive corrections without a learned critic [32]. GRPO-λ introduces eligibility traces and lambda-returns into critic-free RLVR to propagate outcome information backward along the sequence [24]. OTB derives an optimal position-dependent baseline that minimizes per-token gradient variance; the practical estimator sets the baseline at position t to a cumulative-gradient-energy-weighted average of group returns-to-go, where the weights are a causal logit-gradient proxy computed directly from the forward-pass probabilities [18]. PSPO applies potential-based reward shaping theory to construct dense token rewards from outcome signals without altering the optimal policy [13]. Compared with these approaches, our focus is orthogonal: we do not propose a new redistribution rule, but instead diagnose a failure mode that affects essentially all intrinsic-signal-based redistribution methods when applied inside LoRA.

B.5 Alternative Action Granularities and Token Selection

Several papers attack the same underlying problem by changing the optimization unit rather than by redesigning the reward. MA-RLHF introduces macro actions, grouping tokens into higher-level units to shorten the temporal distance between decisions and rewards [4]. GSPO moves importance weighting and clipping from the token level to the sequence level for greater stability in large-scale RL training [43]. These methods reinforce the broader point that the granularity of optimization matters as much as the reward definition.

Outside on-policy RL proper, token-selective optimization has also been explored in adjacent paradigms. ConfPO emphasizes preference-critical tokens based on policy confidence in preference optimization [40], and token weighting for long-range language modeling shows that non-uniform token emphasis can improve optimization even in supervised settings [10]. Together with high-entropy token analyses in RLVR [34], these results strongly suggest that uniform token treatment is an arbitrary design choice rather than a necessity.

C Additional Method Context

C.1 Relationship to Existing Methods

Our formulation recovers standard critic-free RL as a special case: uniform weighting plus a prompt-level batch baseline yields the usual RLOO/GRPO-style estimator. The intrinsic weighting schemes (surprisal, entropy reduction, policy divergence) are the natural baselines within our framework and map cleanly to published methods (e.g., divergence weighting corresponds to the divergence-advantage analyses of *Sparse but Critical* [20], entropy-based weighting to [9, 41]). ARCA is the only scheme we study that uses an explicitly *adaptation-aware* signal. Unlike critic-based PPO, ARCA does not learn a value head. Unlike RED, PRIME, or token-level reward-model methods, ARCA does not construct dense rewards from an auxiliary model. Unlike TEMPO or related tree-based methods, ARCA does not build prefix graphs or estimate branch values. Unlike hard token-selection methods based on top-entropy masking, ARCA retains a dense token update but modulates it continuously using an intrinsic salience score derived from the adapter itself.

The resulting estimator is intentionally minimal. It changes only the within-trajectory allocation of a scalar on-policy advantage, using quantities already available from the forward pass with the adapter disabled. This makes it easy to layer on top of existing critic-free RLVR pipelines while preserving their computational simplicity.

D Theoretical Interpretation

We provide an interpretation of intrinsic token weighting as a variance-control and credit-redistribution mechanism for policy gradient estimation in language models. Our goal is not to derive new convergence guarantees, but to clarify how token weighting relates to advantage estimation and why it can be effective without learning value functions.

D.1 Token Weighting as Credit Redistribution

Consider the standard REINFORCE estimator with a batch-level baseline:

$$g = (R - b) \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(y_t \mid y_{<t}, x). \tag{16}$$

This estimator assigns equal credit to all tokens in a trajectory. Introducing token weights yields:

$$g_w = (R - b) \sum_{t=1}^{T} w_t(y)\, \nabla_\theta \log \pi_\theta(y_t \mid y_{<t}, x), \tag{17}$$

where $\sum_t w_t = 1$.

Importantly, this transformation does not change the reward signal itself; it redistributes how the trajectory-level signal is attributed to individual decisions. When w_t depends only on quantities available at sampling time (e.g., log-probabilities or entropy), the estimator remains on-policy and does not require learning additional functions.

D.2 Relationship to Advantage Estimation

Advantage-based methods can be interpreted as implicitly defining token-level weights via a learned value function:

$$A_t = R - V(y_{<t}, x). \tag{18}$$

In this view, the contribution of each token is scaled according to how much the observed outcome deviates from an estimated expectation conditioned on the prefix.

However, this interpretation relies on the existence of a meaningful value function over prefixes. In language generation, prefixes are non-Markov and do not form a stable state space, making $V(y_{<t}, x)$ difficult to estimate and prone to bias. Token weighting offers an alternative: rather than estimating expected returns from prefixes, it emphasizes tokens based on intrinsic measures of decision salience, such as uncertainty or information gain.

D.3 Connection to Control Variates

From a statistical perspective, subtracting a baseline b is a form of control variate that reduces variance without affecting unbiasedness. Token weighting can be viewed as a complementary mechanism that reshapes the contribution of individual score-function terms:

$$\nabla_\theta \log \pi_\theta(y) = \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(y_t \mid y_{<t}, x). \tag{19}$$

Uniform weighting treats all score-function terms equally, regardless of their variance or relevance. Intrinsic weighting schemes effectively reweight these terms, down-weighting low-variance or low-impact contributions (e.g., high-probability continuation tokens) and emphasizing high-variance or high-impact decisions (e.g., low-probability or uncertainty-reducing tokens). While this reweighting introduces bias relative to the uniform estimator, it can substantially reduce variance, leading to improved optimization dynamics in practice.

D.4 Interpretation of Specific Weighting Schemes

Surprisal Weighting. Surprisal-based weights emphasize tokens with low conditional probability under the current policy. These tokens correspond to decisions where small parameter changes can induce large changes in likelihood, and therefore often dominate gradient variance. Weighting by surprisal concentrates learning signal on such high-sensitivity decisions.

Entropy-Reduction Weighting. Entropy-reduction weighting emphasizes tokens that sharply reduce predictive entropy. These tokens correspond to commitment points in generation, where the model transitions from a diffuse distribution over continuations to a more concentrated one. This mirrors the role of branching points in tree-based reasoning methods, but is computed locally from the model's predictive distribution without explicit tree construction.

D.5 Bias–Variance Tradeoff

Both advantage estimation and intrinsic token weighting introduce bias in exchange for variance reduction. Advantage estimation does so by relying on a learned value function, whose bias can be substantial when the underlying state abstraction is weak. Intrinsic token weighting introduces bias by reshaping the contribution of token-level gradients based on heuristic salience measures. The empirical question is therefore not whether bias is introduced, but whether the bias–variance tradeoff is favorable.

This motivates the empirical comparison implemented in the repository: if intrinsic token weighting improves accuracy, pass@k, or optimization behavior under matched settings, then the bias it introduces may be more benign than the bias induced by ill-defined value targets over text prefixes. The point is not that weighting is unbiased, but that its inductive bias may be better aligned with token-level credit assignment in language generation.

D.6 Signal Collapse Under Low-Rank Adaptation

The preceding analysis assumes that the intrinsic weights $\{w_t\}_{t=1}^T$ are non-degenerate: if every scheme degenerates to $w_t \equiv 1/T$, then token weighting cannot provide any credit redistribution over uniform. We now formalize when this degeneracy occurs under LoRA.

Setup. Let π_{ref} be a frozen base model with last-layer hidden states $h_t^{\text{base}} \in \mathbb{R}^d$ and a LoRA adapter whose net contribution at position t is $\Delta h_t \in \mathbb{R}^d$, giving adapted hidden states $h_t^{\text{adapted}} = h_t^{\text{base}} + \Delta h_t$ and logits $z_t^\theta = W_{\text{lm}} h_t^{\text{adapted}}$. Let $\beta = \max_t \|\Delta h_t\|_2$ denote the maximum per-position adapter contribution and $L = \|W_{\text{lm}}\|_{\text{op}}$ its logit-space Lipschitz constant. Write $G(\cdot)$ for the Gini coefficient of a non-negative vector and $\text{EffN}(w)/T = 1/(T\sum_t w_t^2)$ for the normalized effective-token count.

Proposition 1 (Collapse of surprisal and entropy weighting). *Let* $\alpha_t^{\text{surp}} = -\log \pi_\theta(y_t \mid y_{<t}, x)$ *and* $\alpha_t^{\text{surp,ref}} = -\log \pi_{\text{ref}}(y_t \mid y_{<t}, x)$*, with corresponding normalized weights* w^{surp} *and* $w^{\text{surp,ref}}$*. Then for every trajectory,*

$$\|w^{\text{surp}} - w^{\text{surp,ref}}\|_1 \le \frac{4L\beta}{\sum_t \alpha_t^{\text{surp,ref}}}.$$

In particular, if the base-model surprisal profile is close to uniform (as it is on content-dense reasoning traces with smooth base-model calibration), $w^{\text{surp}} \to 1/T$ *as* $\beta / \min_t \alpha_t^{\text{surp,ref}} \to 0$*. The analogous statement holds for the entropy-reduction score.*

Proof sketch. The softmax Jacobian is 2-Lipschitz in the log-space norm, so the per-token log-probabilities differ by at most $2L\beta$ between the adapted and base model. The projection onto the simplex via within-trajectory normalization is 1-Lipschitz in ℓ_1 up to a factor of $2/\sum_t \alpha_t^{\text{ref}}$; combining these gives the stated bound. $\square$

Proposition 2 (Collapse of divergence weighting). *Let* $\alpha_t^{\text{div}} = |\log \pi_\theta(y_t \mid y_{<t}, x) - \log \pi_{\text{ref}}(y_t \mid y_{<t}, x)|$ *and* $w^{\text{div}} = \alpha^{\text{div}} / \sum_t \alpha_t^{\text{div}}$*. Then*

$$0 \le \alpha_t^{\text{div}} \le 2L\beta \qquad \forall t,$$

so $\max_t \alpha_t^{\text{div}} / \min_t(\alpha_t^{\text{div}} + \varepsilon) \to 1$ *as* $\beta \to 0$*, and consequently* $G(w^{\text{div}}) \to 0$ *and* $\text{EffN}(w^{\text{div}})/T \to 1$*. That is, divergence weighting collapses to the uniform distribution whenever the LoRA adapter's per-token perturbation norm vanishes uniformly.*

The key observation is that the intrinsic scores are all determined by the per-token *log-probability differential*, which is precisely what LoRA's rank-r constraint drives small and approximately uniform across positions. The collapse is not a property of a particular weighting scheme; it is a property of measuring per-token variation through the output distribution when the policy has been constrained to a small neighbourhood of the reference.

D.7 ARCA and Position-Discriminative Signal

ARCA side-steps the collapse by measuring a quantity that is position-varying even when the adapter's logit-space impact is small.

Proposition 3 (ARCA remains non-degenerate). *Suppose the adapter-induced residual admits the decomposition* $\Delta h_t = \sum_\ell B_\ell A_\ell x_{\ell,t}^{\text{pre}} + e_t$*, where* $x_{\ell,t}^{\text{pre}}$ *is the pre-adapter activation at layer* ℓ *and position* t *and* $\|e_t\|$ *is a higher-order cross-layer term. Let* $v_{\ell,t} = B_\ell A_\ell x_{\ell,t}^{\text{pre}}$ *and define* $V_\ell = \text{Var}_t(\|v_{\ell,t}\|)$ *and* $\bar{V}_\ell = \mathbb{E}_t[\|v_{\ell,t}\|]$*. Then*

$$\text{Var}_t(\alpha_t^{\text{ARCA}}) \ge \max_\ell V_\ell - O\left(\sum_\ell \bar{V}_\ell \, \|e_t\|\right).$$

In particular, as long as the pre-adapter activations $x_{\ell,t}^{\text{pre}}$ *have non-trivial position-to-position variance at any layer,* α_t^{ARCA} *has bounded-below variance across positions, and its normalized weights* w_t^{ARCA} *do not converge to the uniform distribution as the adapter shrinks uniformly.*

The intuitive content of Proposition 3 is that ARCA inherits its position-discrimination from the *base model's own activation pattern* (which is already non-uniform because attention and layer norm routing produce content-versus-filler distinctions) rather than from a quantity that LoRA is specifically constrained to make small. Scaling the adapter weights uniformly scales α^{ARCA} uniformly, so the normalized weights w^{ARCA} are scale-invariant in the adapter magnitude.

D.8 Bias–Variance Tradeoff

Both advantage estimation and intrinsic token weighting introduce bias in exchange for variance reduction. Advantage estimation does so by relying on a learned value function, whose bias can be substantial when the underlying state abstraction is weak. Intrinsic token weighting introduces bias by reshaping the contribution of token-level gradients based on heuristic salience measures. The empirical question is therefore not whether bias is introduced, but whether the bias–variance tradeoff is favorable—and, per Propositions 1 and 2, whether the bias-inducing signal survives the LoRA bottleneck at all.

D.9 Summary

This perspective reframes advantage estimation as one particular approach to credit redistribution in policy gradient methods. Intrinsic token weighting offers an alternative that leverages the structure of the language model's predictive distribution, without assuming the existence of meaningful state values. Under LoRA, however, the output-distribution-based signals that most published weighting methods rely on are driven toward uniform; the adapter-residual signal that ARCA uses is, by Proposition 3, provably non-degenerate in the same regime.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes].

   Justification: The abstract and introduction frame ARCA as a mechanism for testing whether adapter-residual token credit remains non-degenerate under LoRA, and the experiments section limits the empirical scope to the seven-run MATH/Qwen3-1.7B sweep.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes].

   Justification: Section 3 includes a limitations subsection stating that the current sweep uses one model, one dataset, one seed, one advantage estimator, and seven configurations.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [No].

Justification: Section D states the assumptions and gives proof sketches for the collapse and non-degeneracy propositions, but the current submission does not include full formal proofs for every theoretical statement.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes].

Justification: Section 3 specifies the model, task, reward, advantage estimator, LoRA ranks, rollout budget, evaluation size, seed, optimizer, and decoding hyperparameters used for the reported seven-run sweep.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: **[TODO]**.

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes].

   Justification: Section 3 describes the MATH train/test setting, prompt format, reward, optimizer, learning-rate schedule, LoRA ranks, rollout count, decoding parameters, checkpointing, and final evaluation metrics.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No].

   Justification: The submission-time sweep uses a single seed and does not report confidence intervals or significance tests. The limitations subsection explicitly notes that the results do not establish robustness across seeds, tasks, or algorithms.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes].

   Justification: Section 3 states that each run fits on a single H100-class GPU, that the seven reported configurations are independent single-GPU runs, and that preliminary development runs are not part of the reported comparison.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: **[TODO]**.

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes].

    Justification: Section 3 includes a broader impacts subsection discussing potential positive effects on sample efficiency and interpretability, as well as potential misuse in harmful or misleading fine-tuning settings.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A].

Justification: The paper does not release a new high-risk general-purpose pretrained model or scraped dataset; the reported work uses existing public model and dataset assets for a methodological study.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: **[TODO]**.

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.

- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: **[TODO]**.

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A].

    Justification: The paper does not involve crowdsourcing experiments or research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A].

    Justification: The paper does not involve crowdsourcing experiments or research with human subjects, so IRB or equivalent human-subjects review is not applicable.

    Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes].

    Justification: Large language models are central to the research object and methodology; the paper identifies the model family, LoRA fine-tuning setting, reward construction, and generation/evaluation procedure in Section 3.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.